

Основи API 2



SOAP API

Вступ до SOAP API

- API SOAP (Simple Object Access Protocol) слугують основою у сфері веб-сервісів, сприяючи надійній комунікації між різними системами. У цій презентації ми розкриємо основні характеристики та функціональні можливості API SOAP, проливаючи світло на їхню незмінну важливість в умовах постійного розвитку технологічних ландшафтів.

Історія SOAP

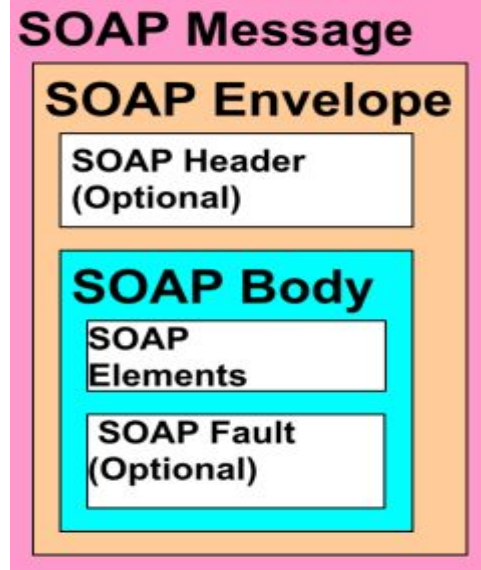
- Розроблено Microsoft наприкінці 1990-х років.
- Спочатку задумувався як протокол для обміну структурованою інформацією в децентралізованому, розподіленому середовищі.
- Розвинувся в широко прийнятий стандарт для зв'язку веб-сервісів.
- Стандартизований Консорціумом Всесвітньої павутини (W3C) та прийнятий різними галузями.
- Продовжує служити фундаментальною технологією для інтеграції та сумісності підприємств.

Розгадування SOAP: визначення та ключові характеристики

- SOAP (Simple Object Access Protocol) – це протокол для обміну структурованою інформацією під час реалізації веб-сервісів.
- Він використовує XML (Extensible Markup Language) для формату повідомлень та працює через різні транспортні протоколи, включаючи HTTP, SMTP та інші.
- Ключові характеристики включають незалежність від платформи та мови, що забезпечує сумісність між різними системами.
- Повідомлення SOAP зазвичай складаються з конверта, заголовка та тіла, що дозволяє інкапсулювати дані та метадані у стандартизованому форматі.
- SOAP сприяє обміну структурованою інформацією між програмами, забезпечуючи безперешкодну інтеграцію та зв'язок у розподілених середовищах.

Деконструкція SOAP-повідомлень

- **Конверт:** Зовнішній елемент, що містить усе SOAP-повідомлення. Визначає XML-документ як SOAP-повідомлення та інкапсулює заголовок і тіло.
- **Заголовок:** Необов'язковий елемент в конверті, який містить інформацію, специфічну для програми, таку як облікові дані автентифікації або дані маршрутизації.
- **Тіло:** Обов'язковий елемент в конверті, що містить фактичне корисне навантаження або дані, що передаються між клієнтом і сервером.
- **Помилка:** Необов'язковий елемент в конверті, що використовується для передачі інформації про помилку та стан у разі збою або винятку під час обробки повідомлення.



Роль XML у SOAP



- SOAP використовує XML (розширювану мову розмітки) як базовий формат повідомлень. XML забезпечує стандартизований та незалежний від платформи спосіб структурування даних, що є важливим для сумісності між різними системами. Повідомлення SOAP кодуються в XML, використовуючи його ієрархічну структуру для представлення складних типів даних. Такі елементи, як конверт, заголовок, тіло та помилка, визначаються за допомогою XML, що дозволяє передавати структуровану інформацію між клієнтами та серверами. Схеми XML можна застосовувати для забезпечення типізації та перевірки даних, забезпечуючи цілісність повідомлень SOAP, що обмінюються в розподілених системах.

SOAP проти REST: порівняльний огляд

SOAP та REST представляють різні підходи до проектування API:

- SOAP — це важкий протокол, який спирається на XML та пропонує широкий функціонал, що підходить для складних інтеграцій.
- REST — це легкий протокол, який використовує простіші формати, такі як JSON, та зосереджений на масштабованості та простоті, що ідеально підходить для веб- та мобільних додатків. Вибираючи між ними, враховуйте такі фактори, як складність проекту та вимоги до продуктивності.



SOAP та транспортні протоколи

SOAP, будучи універсальним, адаптується до кількох транспортних протоколів:

- HTTP: Зазвичай використовується для веб-зв'язку, що дозволяє надсилати SOAP-повідомлення через Інтернет.
- SMTP: Використовується для передачі електронної пошти, що дозволяє надсилати SOAP-повідомлення електронною поштою.
- Інше: SOAP не обмежується HTTP та SMTP; він працює через такі протоколи, як FTP та TCP, забезпечуючи гнучкість у різноманітних мережевих середовищах. Ця універсальність дозволяє SOAP безперешкодно інтегруватися в різні системи, задовольняючи різні вимоги до зв'язку.

Функції безпеки SOAP

- SOAP реалізує надійні заходи безпеки, головним чином за допомогою таких стандартів, як WS-Security, що забезпечують цілісність повідомлень, конфіденційність, автентифікацію та авторизацію. Додаткові стандарти, такі як WS SecureConversation, WS-Trust та WS-Policy, ще більше підвищують безпеку, сприяючи безпечним розмовам, управлінню токенами та вираженню політик. Разом ці заходи створюють комплексну систему безпеки для веб-сервісів на основі SOAP.

Розуміння WSDL

- WSDL (мова опису веб-сервісів) відіграє життєво важливу роль у веб-сервісах на основі SOAP. Вона служить для опису інтерфейсу та операцій сервісу, дозволяючи клієнтам зрозуміти, як з ним взаємодіяти. Написані на XML, документи WSDL описують кінцеві точки сервісу, операції, повідомлення та типи, що забезпечує безперешкодну інтеграцію та сумісність між системами.

```
<?xml version="1.0" encoding=
<definitions name="AktienKurs
  targetNamespace="http://loc
  xmlns:xsd="http://schemas.xmlsoap.or
  xmlns="http://schemas.xmlsoap.org/wsdl
  <service name="AktienKurs">
    <port name="AktienSoapPort" binding
      <soap:address location="http://loc
    </port>
    <message name="Aktie.HoleWert">
      <part name="body" element="xsd:Tra
    </message>
    ...
  </service>
</definitions>
```

WSDL

Розуміння динаміки зв'язування SOAP

- Зв'язування SOAP є ключовим для оркестрування передачі та форматування повідомлень SOAP через різні протоколи, такі як HTTP та SMTP. Визначаючи базовий протокол для обміну повідомленнями та вказуючи структуру повідомлення в ньому, зв'язування SOAP забезпечує адаптивність та стандартизацію. Цей механізм дозволяє веб-сервісам на основі SOAP безперешкодно взаємодіяти в гетерогенних мережевих середовищах, сприяючи взаємодії між системами з різними архітектурами та технологіями. По суті, зв'язування SOAP служить основою універсальності SOAP, сприяючи безперебійній взаємодії в розподілених обчислювальних середовищах.

Дії SOAP

- **Визначення:** Дії SOAP розмежовують окремі операції в межах SOAP-сервісів.
- **Керівництво:** Вони служать навігаційними маркерами для обміну повідомленнями між клієнтами та сервісами.
- **Відображення функціональності:** Кожна дія SOAP відповідає певній функціональності або методу, що надається сервісом.
- **Інтерактивність:** Дії SOAP, що мають вирішальне значення для клієнтів, сприяють безперебійній взаємодії, надаючи чіткі точки доступу до функціональності сервісу.

Запит SOAP

- Визначення конверта: Почніть з конверта SOAP, що містить повідомлення запиту.
- Складання заголовка (необов'язково): Включіть будь-яку додаткову інформацію, таку як токени автентифікації.
- Створення тіла: Заповніть тіло потрібним викликом методу та параметрами.
- Обертання в конверт SOAP: Інкапсуляція заголовка та тіла в конверт SOAP.
- Надсилання запиту: Передача запиту SOAP до кінцевої точки сервера SOAP.
- Отримання відповіді: Очікування та обробка відповіді SOAP, повернутої сервером.

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ser="http://soap-service.exxspes.space/Service">
  <soapenv:Header>
    <authToken>*AUTH TOKEN*</authToken>
  </soapenv:Header>
  <soapenv:Body>
    <ser:MethodName>
      <param1>value1</param1>
      <param2>value2</param2>
    </ser:MethodName>
  </soapenv:Body>
</soapenv:Envelope>
```

Обробка відповіді SOAP

- Під час обробки відповіді SOAP вкрай важливо ефективно розбиратися вилучення потрібної інформації. Зазвичай це передбачає використання XML, таких як `xml.etree.ElementTree` у Python або аналогічних інструментів в інших мовах програмування. Навігація по структурі XML за допомогою виразів XPath дозволяє розробникам знаходити певні елементи у відповіді SOAP та вилучати відповідні дані для подальшої обробки або представлення.

Здійснення викликів SOAP API

- Для здійснення викликів SOAP API розробники часто використовують спеціалізовані інструменти, такі як Postman або SoapUI. Ці інструменти надають зручні інтерфейси для створення SOAP-запитів, включаючи визначення кінцевої точки SOAP, заголовків та тіла запиту. Потім користувачі можуть надіслати запит та отримати SOAP-відповідь безпосередньо в інструменті, що спрощує процес тестування та налагодження SOAP API. Крім того, ці інструменти пропонують такі функції, як автоматична генерація зразків запитів, перевірка відповідей та підтримка різних методів автентифікації, що підвищує ефективність розробки та тестування SOAP API.

Найкращі практики для SOAP API

- **Версіонування:** Впроваджуйте версіонування для підтримки зворотної сумісності та забезпечення поступового оновлення.
- **Обробка помилок:** Визначайте чіткі повідомлення про помилки та коди для полегшення усунення несправностей та налагодження.
- **Безпека:** Використовуйте стандарти WS-Security для автентифікації, шифрування та перевірки цілісності.
- **Документація:** Надайте вичерпну документацію, що охоплює кінцеві точки API, методи, параметри та очікувані відповіді.
- **Оптимізація:** Мінімізуйте розмір повідомлень та оптимізуйте продуктивність, зменшуючи непотрібні передачі даних.
- **Тестування:** Проводьте ретельне тестування всіх функцій API, включаючи граничні випадки та сценарії помилок.
- **Моніторинг:** Впроваджуйте механізми моніторингу для відстеження використання API, показників продуктивності та потенційних проблем у режимі реального часу.
- **Відповідність:** Дотримуйтесь стандартів та рекомендацій SOAP для забезпечення сумісності з клієнтами та іншими системами.
- **Механізм зворотного зв'язку:** Встановіть цикл зворотного зв'язку зі споживачами API для збору інформації та вдосконалення API з часом.
- **Виведення з експлуатації версій:** Чітко повідомляйте терміни виведення з експлуатації старіших версій API та надавайте шляхи міграції для споживачів.

Переваги використання SOAP

- SOAP забезпечує надійність, безпеку та стандартизацію. Він гарантує доставку та цілісність повідомлень, включає надійну автентифікацію та шифрування, а також дотримується галузевих стандартів сумісності. Розширюваність SOAP, підтримка інструментів, незалежність від протоколів та широке впровадження ще більше підвищують його привабливість у корпоративних середовищах.



Проблеми з SOAP

- **Накладні витрати даних:** Повідомлення SOAP можуть бути багатослівними через форматування XML, що призводить до збільшення використання пропускної здатності та зниження продуктивності.
- **Складність інтеграції:** Впровадження та обслуговування систем на основі SOAP може вимагати спеціалізованих знань та інструментів, що потенційно збільшує час розробки та витрати.

SOAP API у реальних проектах

- Amazon Web Services (AWS): AWS пропонує SOAP API для різних сервісів, таких як Amazon Simple Storage Service (S3) та Amazon Simple Queue Service (SQS), що дозволяє розробникам безпечно інтегрувати функції AWS у свої програми.
- Microsoft Dynamics CRM: Microsoft Dynamics CRM надає SOAP API для безперешкодної інтеграції з іншими продуктами Microsoft та системами сторонніх розробників, що сприяє ефективному управлінню взаємовідносинами з клієнтами в організаціях.
- SAP Business Suite: SAP пропонує SOAP API для своїх програм Business Suite, що дозволяє підприємствам інтегрувати функції SAP у свої бізнес-процеси, такі як SAP ERP, SAP CRM та SAP Business Intelligence.
- API Google AdWords: Google AdWords надає SOAP API для програмного доступу до даних AdWords та управління рекламними кампаніями, що дозволяє рекламодавцям автоматизувати завдання управління кампаніями та звітності.

Майбутнє SOAP API

- Забігаючи вперед, SOAP API готові розвиватися кількома ключовими напрямками. Хоча REST API здобули популярність, SOAP збережеться, особливо в корпоративних середовищах, які цінують надійність та безпеку. Зусилля щодо модернізації SOAP, такі як підтримка корисних навантажень JSON та оптимізовані формати повідомлень, підвищать його продуктивність та досвід розробників, забезпечуючи його подальшу актуальність. Крім того, SOAP API будуть інтегровані з новими технологіями, такими як мікросервіси та хмарні нативні додатки, адаптуючись до змінних парадигм розробки. Такі галузі, як фінанси, охорона здоров'я та уряд, що надають пріоритет безпеці та сумісності, збережуть залежність від SOAP для критично важливих додатків. Крім того, зобов'язання SOAP щодо стандартизації та сумісності забезпечить безперебійну комунікацію між різними системами та платформами, закріплюючи його роль у майбутньому веб-сервісів.

SOAP та сучасні альтернативи

- У сучасному технологічному ландшафті SOAP співіснує із сучасними альтернативами, такими як REST, GraphQL та gRPC. Хоча SOAP залишається поширеним у корпоративних середовищах, що вимагають надійності та безпеки, новіші технології пропонують інші компроміси. Простота та гнучкість REST роблять його популярним для веб-API, тоді як GraphQL перевершує інших у забезпеченні точного пошуку даних у взаємодії клієнт-сервер. gRPC, зосереджуючись на високій продуктивності та мовно-незалежній комунікації, набирає обертів в архітектурах мікросервісів. Незважаючи на ці альтернативи, SOAP зберігає свої позиції в таких галузях, як фінанси та охорона здоров'я, де суворі стандарти безпеки та сумісності є першочерговими. З розвитком технологій організації продовжуватимуть оцінювати придатність SOAP та альтернативних підходів на основі своїх конкретних вимог та варіантів використання.

Інструменти для розробки за допомогою SOAP

- **Apache Axis:** Широко використовуваний SOAP-рушій для Java, що надає інструменти для створення, розгортання та керування веб-сервісами SOAP.
- **.NET Framework:** Microsoft .NET Framework містить вбудовану підтримку для розробки веб-сервісів на основі SOAP за допомогою ASP.NET та Windows Communication Foundation (WCF).
- **SOAPUI:** Комплексний інструмент тестування, який підтримує SOAP та REST API, пропонуючи функції для функціонального тестування, тестування навантаження та тестування безпеки SOAP-сервісів.
- **Postman:** Популярна платформа розробки API, що підтримує SOAP-запити, що дозволяє розробникам легко створювати, надсилати та аналізувати SOAP-повідомлення.
- **Python Zeep:** Клієнтська бібліотека SOAP для Python, що пропонує прості та інтуїтивно зрозумілі API для взаємодії з SOAP-сервісами та обробки SOAP-повідомлень.
- **Spring Web Services:** Фреймворк для створення SOAP-веб-сервісів на Java, що забезпечує підтримку розробки на основі контрактів та інтеграцію з екосистемою Spring.



GraphQL

Вступ до



GraphQL

- GraphQL — це мова запитів для API, розроблена Facebook у 2012 році та випущена як проект з відкритим кодом у 2015 році. На відміну від традиційних REST API, які надають фіксований набір кінцевих точок, що повертають попередньо визначені структури даних, GraphQL дозволяє клієнтам запитувати у сервера саме ті дані, які їм потрібні. Вона була розроблена для усунення обмежень REST API, таких як надмірна вибірка, недостатня вибірка та відсутність гнучкості у пошуку даних. За допомогою GraphQL клієнти можуть запитувати лише ті конкретні поля та вкладені дані, які їм потрібні, що призводить до ефективнішої вибірки даних та зменшення накладних витрат мережі.

Потреба в GraphQL

- GraphQL усуває обмеження традиційних REST API, включаючи надмірну вибірку, недостатню вибірку, фіксовані структури даних, проблеми керування версіями та складні запити. Він надає гнучку мову запитів та середовище виконання для виконання запитів, що дозволяє ефективніше та адаптованіше отримувати дані для клієнтських застосунків.

Основні концепції GraphQL

- **Запити:** Запити використовуються клієнтами для запиту даних з GraphQL API. Вони вказують потрібні поля та вкладені структури даних, що дозволяє точно отримувати дані.
- **Мутації:** Мутації використовуються для зміни даних на сервері. Вони дозволяють клієнтам виконувати операції створення, оновлення та видалення, забезпечуючи узгодженість та цілісність даних.
- **Підписки:** Підписки дозволяють клієнтам отримувати оновлення в режимі реального часу від сервера. Вони встановлюють постійне з'єднання між клієнтом і сервером, що дозволяє здійснювати push-повідомлення для таких подій, як надходження нових даних або зміни.

Розуміння запитів GraphQL

- Запити GraphQL структуровані ієрархічно, що нагадує форму запитуваних даних. Вони починаються з ключового слова запиту, за яким йде назва операції (необов'язково) та набір полів, що вказують на дані, які потрібно отримати. Клієнти вказують потрібні їм поля, включаючи аргументи для фільтрації або сортування, і можуть запитувати вкладені поля для проходження складних структур даних. Таке точне запитування дозволяє ефективно отримувати дані, зменшуючи надмірну та недостатню вибірку, що є типовим для традиційних REST API.

```
query GetBookInfo {  
  book(id: "1") {  
    title  
    publicationYear @include(if: true)  
    author @skip(if: false) {  
      name  
      birthYear  
    }  
  }  
}
```

Розуміння підписок GraphQL

- Підписки GraphQL дозволяють оновлювати дані між клієнтами та серверами в режимі реального часу. На відміну від традиційних REST API, які вимагають від клієнтів періодичного опитування щодо оновлень, підписки встановлюють постійне з'єднання між клієнтом і сервером. Коли відбуваються певні події або дані змінюються, сервер надсилає оновлення підписаним клієнтам у режимі реального часу. Ця можливість є критично важливою для програм, що потребують оновлень у реальному часі, таких як програми обміну повідомленнями, стрічки новин у реальному часі та інструменти для спільної роботи, покращуючи взаємодію з користувачем та інтерактивність без необхідності постійного опитування.

Система типів у GraphQL

- **Скаляри:** Скаляри представляють примітивні типи даних, такі як String, Int, Float, Boolean та ID. Вони є атомарними значеннями та не можуть мати підполів.
- **Об'єкти:** Об'єкти представляють складні структури даних з кількома полями. Кожне поле може бути скалярного або об'єктного типу, що дозволяє вкладені структури даних.
- **Переліки:** Переліки визначають набір можливих значень для поля. Клієнти можуть вибирати значення лише з визначеного набору, забезпечуючи узгодженість даних та валідацію.
- **Інтерфейси:** Інтерфейси визначають спільний набір полів, що використовуються кількома типами об'єктів. Вони дозволяють виконувати поліморфні запити, де один запит може повертати різні типи об'єктів, що реалізують інтерфейс.

Визначення схеми в GraphQL

- У GraphQL схема визначає доступні дані та операції, які можна виконувати з цими даними. Вона включає типи (такі як користувачі та повідомлення), запити (для отримання даних), мутації (для зміни даних) та, за бажанням, підписки (для оновлень у режимі реального часу). Ця схема діє як контракт між клієнтами та серверами, що дозволяє точно визначити, які дані можна отримати та як їх можна змінити в GraphQL API.

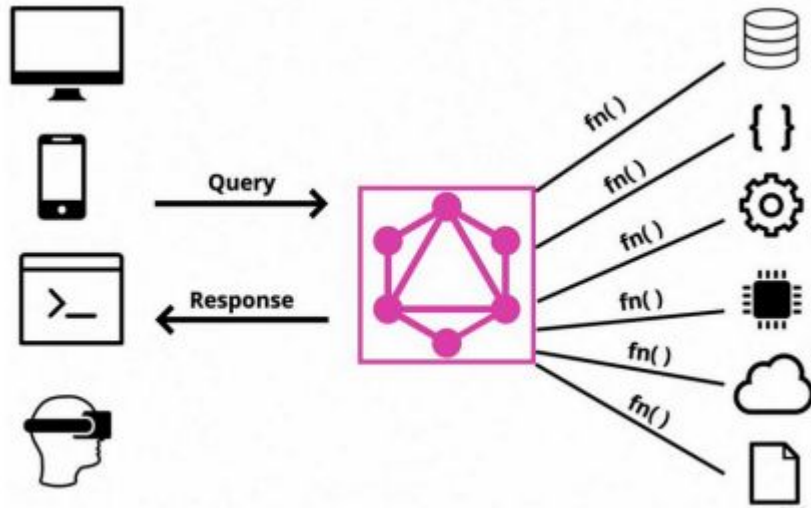
```
{  
  hero {  
    name  
    friends {  
      name  
      homeWorld {  
        name  
        climate  
      }  
      species {  
        name  
        lifespan  
        origin {  
          name  
        }  
      }  
    }  
  }  
}
```

```
type Query {  
  hero: Character  
}  
  
type Character {  
  name: String  
  friends: [Character]  
  homeWorld: Planet  
  species: Species  
}  
  
type Planet {  
  name: String  
  climate: String  
}  
  
type Species {  
  name: String  
  lifespan: Int  
  origin: Planet  
}
```

Резолвери в GraphQL

- Резолвери діють як мости між схемою GraphQL та джерелами даних. Вони отримують запитувані дані на основі запитів клієнта та зіставляють їх з відповідними полями схеми. Резолвери забезпечують динамічне отримання даних та підключаються до різних джерел даних для ефективного отримання. Їхня роль забезпечує безперебійний зв'язок між схемою та базовими даними, сприяючи гнучкому та ефективному отриманню даних в API GraphQL.

Архітектура API GraphQL



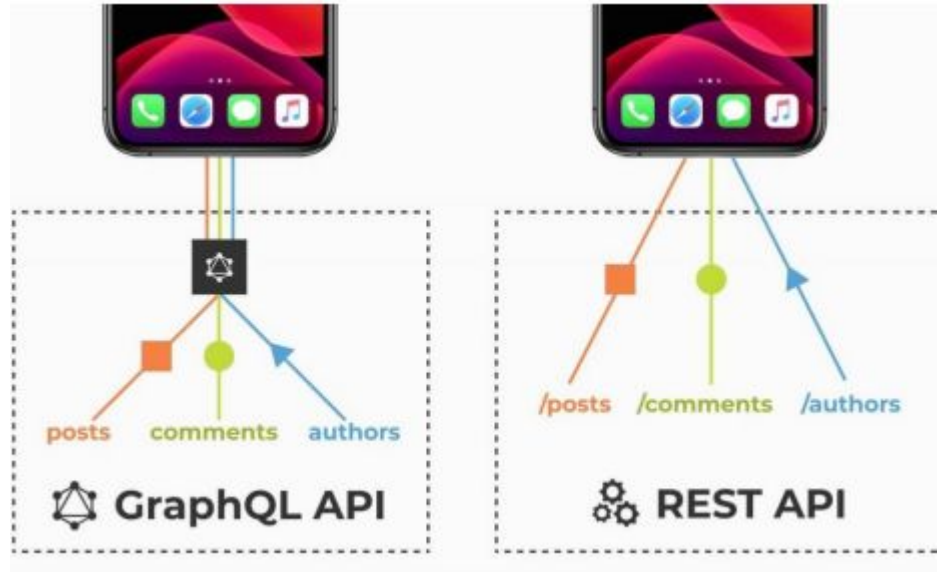
GraphQL вписується в архітектуру сервер-клієнт як мова запитів та середовище виконання для API. У цій архітектурі клієнт надсилає запити GraphQL на сервер, вказуючи точні вимоги до даних. Сервер, оснащений реалізацією GraphQL, обробляє ці запити, переводить їх у відповідні джерела даних за допомогою резолверів та повертає запитувані дані клієнту. Ця архітектура забезпечує більш гнучкий та ефективний підхід до отримання даних порівняно з традиційними REST API, оскільки клієнти можуть запитувати саме ті дані, які їм потрібні, без надмірного або недостатнього отримання. Крім того, система типів та схема GraphQL забезпечують міцний контракт між клієнтом та сервером, забезпечуючи передбачувану взаємодію з даними та сприяючи ефективній розробці та співпраці.

Переваги GraphQL

- Ефективне завантаження даних: Дозволяючи клієнтам запитувати лише ті дані, які їм потрібні, GraphQL зменшує надмірну та недостатню вибірку, що призводить до оптимізованого використання мережі та швидшого часу відгуку.
- Оптимізована архітектура: Завдяки єдиній кінцевій точці та гнучким запитам GraphQL спрощує керування API та зменшує потребу в управлінні версіями, що призводить до більш зручної в обслуговуванні та масштабованої архітектури.
- Індивідуальні запити: Клієнти мають точний контроль над формою та структурою даних, які вони отримують, що дозволяє створювати індивідуальні запити для конкретних випадків використання. Ця гнучкість підвищує продуктивність розробників та покращує взаємодію з користувачем, надаючи саме ті дані, які їм потрібні.

GraphQL проти REST

GraphQL та REST принципово відрізняються за отриманням даних та гнучкістю. GraphQL дозволяє точно отримувати дані за допомогою динамічних запитів, зменшуючи надмірне та недостатнє отримання. Він відрізняється ефективністю та гнучкістю, що підходить для складних програм, керованих даними. REST спирається на фіксовані кінцеві точки, що призводить до потенційного надмірного отримання та обмеженої гнучкості. Він залишається надійним для простіших випадків використання та сумісності з існуючими системами. Вибір залежить від потреб проекту, причому GraphQL пропонує ефективність та адаптивність, тоді як REST забезпечує простоту та знайомство.



Обробка помилок у GraphQL

- **Відповіді на помилки:** GraphQL надає детальні відповіді на помилки у форматі JSON, включаючи зручні повідомлення, коди помилок та шляхи помилок у запиті.
- **Часткові результати:** Запити можуть повертати часткові результати навіть за наявності помилок, що дозволяє клієнтам отримувати коректні дані для успішно виконаних частин запиту.
- **Обробка на стороні клієнта:** Клієнти можуть реалізовувати стратегії обробки помилок на основі наданої інформації про помилки, такі як відображення повідомлень про помилки або повторна спроба невдалих запитів.

Безпека в GraphQL

- Викриття даних: Запити можуть ненавмисно розкривати конфіденційні дані, що вимагає ретельної перевірки даних для запобігання несанкціонованому доступу.
- Складність запитів: Складні запити можуть призвести до проблем із продуктивністю або атак типу «відмова в обслуговуванні», що вимагає обмеження глибини та складності запитів.
- Автентифікація та авторизація: Надійні механізми автентифікації та авторизації є життєво важливими для контролю доступу до конфіденційних даних та операцій, гарантуючи, що лише авторизовані користувачі можуть виконувати дозволені дії.
- Перевірка вхідних даних: Комплексна перевірка вхідних даних є важливою для запобігання атакам ін'єкцій та підтримки цілісності даних.

cache: {

Кешування за допомогою GraphQL

Кешування в GraphQL є нюансованим через динамічні запити та різноманітні форми даних. Проблеми включають гранулярність, ефективність та анулювання кешу. Рішення включають адаптовані підходи до кешування, нормалізацію та часткове кешування для оптимізації продуктивності та вирішення цих складнощів.

Створення GraphQL API

Створення GraphQL API включає кілька кроків:

1. Визначення схеми: Визначте типи, запити, мутації та підписки у схемі GraphQL.
2. Реалізація резолвера: Реалізуйте функції резолвера для отримання даних для кожного поля у схемі.
3. Інтеграція джерел даних: Підключення до джерел даних, таких як бази даних або зовнішні API, для отримання даних у функціях резолвера.
4. Аутентифікація та авторизація: Реалізуйте механізми автентифікації та авторизації для контролю доступу до API.
5. Тестування: Напишіть модульні тести, щоб переконатися, що API функціонує належним чином, та перевірити відповіді на запити.
6. Документація: Задokumentуйте схему API, запити, мутації та підписки, щоб допомогти розробникам використовувати API.
7. Розгортання: Розгорніть GraphQL API на хостинг-провайдера або сервер для використання у виробничому середовищі.

Питання та відповіді